

Risky Behavior

Sure it's risky,
but I can **handle** it if
something goes wrong.



MIDI file

play high C,
hit it
and hold it
for 2 beats

MIDI file has information about how a song should be played, but it doesn't have any actual sound data. It's kind of like sheet music instructions for a player-piano.



MIDI device knows how to "read" a MIDI file and play back the sound. The device might be a synthesizer keyboard or some other kind of instrument.

Usually, a MIDI instrument can play a LOT of different sounds (piano, drums, violin, etc.), and all at the same time. So a MIDI file isn't like sheet music for just one musician in the band—it can hold the parts for ALL the musicians playing a particular song.

```
import javax.sound.midi.*;
```

← import the javax.sound.midi package

```
public class MusicTest1 {
```

```
    public void play() {
```

```
        try {
```

```
            Sequencer sequencer = MidiSystem.getSequencer();
```

```
            System.out.println("Successfully got a sequencer");
```

```
        }
```

```
    }
```

```
public static void main(String[] args) {
```

```
    MusicTest1 mt = new MusicTest1();
```

```
    mt.play();
```

```
}
```

```
}
```

We need a Sequencer object. It's the main part of the MIDI device/instrument we're using. It's the thing that, well, sequences all the MIDI information into a "song." But we don't make a brand new one ourselves—we have to ask the MidiSystem to give us one.

Something's wrong!

This code won't compile! The compiler says there's an "unreported exception" that must be caught or declared.



Dr. Grace Hopper

92

9/9

0800 Antan started

1000 " stopped - antan ✓

1300 (032) MP-MC ~~1.982647000~~ ~~2.130476415~~ { 1.2700 · 9.037847025
9.037846795 correct

(033) PRO 2 2.130476415

correct 2.130676415

Relays 6-2 in 033 failed special speed test
in relay " 10,000 test.

Relays changed

1700 Started Cosine Tape (Sine check)

1525 Started Multi-Adder Test.

1545  Relay #70 Panel F
(moth) in relay.

First actual case of bug being found.

1630 Antan started.

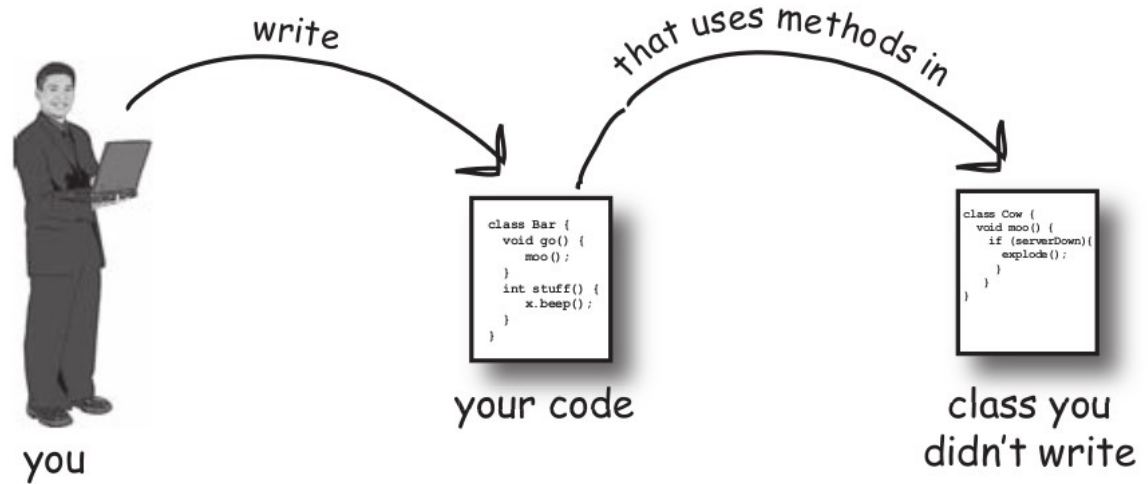
1700 closed down.

Relay 3145
Relay 3370

First computer bug, September 9 1947

What happens when a method you want to call (probably in a class you didn't write) is risky?

- 1 Let's say you want to call a method in a class that you didn't write.



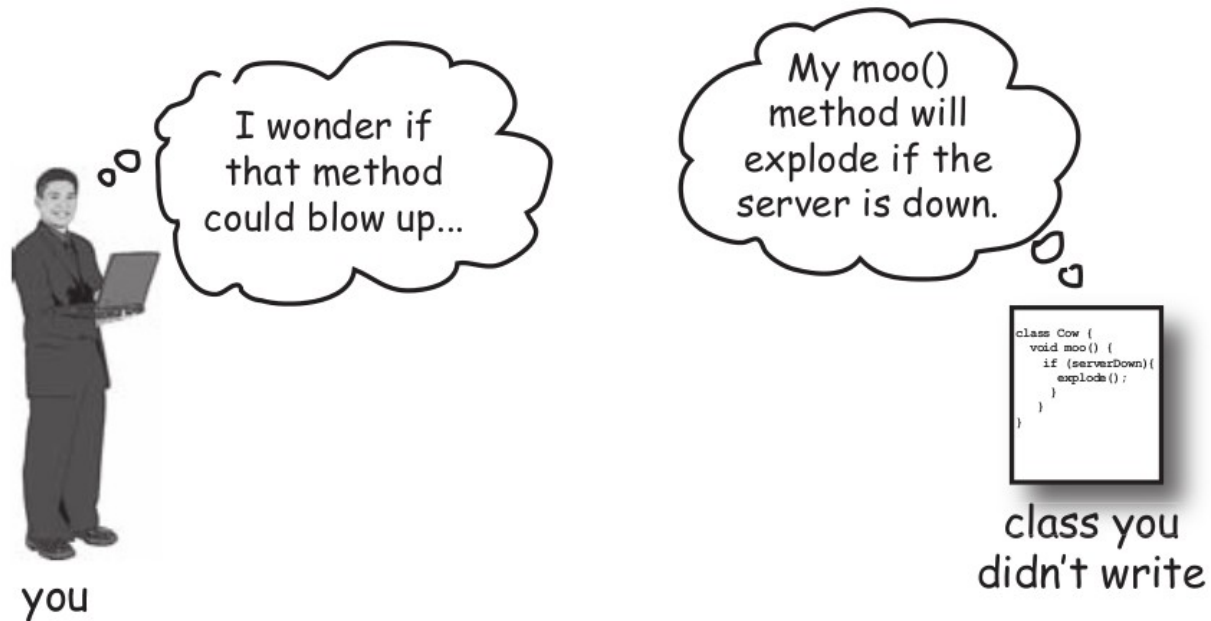
- ② That method does something risky, something that might not work at runtime.

```
class Cow {  
    void moo() {  
        if (serverDown) {  
            explode();  
        }  
    }  
}
```

class you didn't write

```
void moo() {  
    if (serverDown) {  
        explode();  
    }  
}
```

- ③ You need to *know* that the method you're calling is risky.





- ④ **You then write code that can handle the failure if it *does* happen. You need to be prepared, just in case.**

you

Now that I know, I can take precautions.

write safely



```
class Bar {  
    void go() {  
        try{  
            moo();  
        }catch(MX m){  
            cry();  
        }  
    }  
}
```

your code

getSequencer

```
public static Sequencer getSequencer()  
                        throws MidiUnavailableException
```



Obtains the default Sequencer, connected to a default device. The returned Sequencer instance is connected to the default Synthesizer, as returned by `getSynthesizer()`. If there is no Synthesizer available, or the default Synthesizer cannot be opened, the sequencer is connected to the default Receiver, as returned by `getReceiver()`. The connection is made by retrieving a Transmitter instance from the Sequencer and setting its Receiver. Closing and re-opening the sequencer will restore the connection to the default device.

This method is equivalent to calling `getSequencer(true)`.

If the system property `javax.sound.midi.Sequencer` is defined or it is defined in the file "sound.properties", it is used to identify the default sequencer. For details, refer to the [class description](#).

Returns:

the default sequencer, connected to a default Receiver

Throws:

MidiUnavailableException - if the sequencer is not available due to resource restrictions, or there is no Receiver available by any installed `MidiDevice`, or no sequencer is installed in the system

The API does tell you that `getSequencer()` can throw an exception: `MidiUnavailableException`. A method has to declare the exceptions it might throw.

Dear Compiler,

I know I'm taking a risk here, but
don't you think it's worth it? What
should I do?

signed, geeky in Waikiki

Dear geeky,

Life is short (especially on the heap).
Take the risk. *try* it. But just in case
things don't work out, be sure to *catch*
any problems before all hell breaks loose.

```
import javax.sound.midi.*;
```

```
public class MusicTest1 {
```

```
    public void play() {
```

```
        try {
```

```
            Sequencer sequencer = MidiSystem.getSequencer();
```

```
            System.out.println("Successfully got a sequencer");
```

```
        } catch (MidiUnavailableException e) {
```

```
            System.out.println("Bummer");
```

```
        }
```

```
    }
```

```
    public static void main(String[] args) {
```

```
        MusicTest1 mt = new MusicTest1();
```

```
        mt.play();
```

```
    }
```

```
}
```

Put the risky thing in a "try" block. It's the "risky" getSequencer method that might throw an exception.

Make a "catch" block for what to do if the exceptional situation happens—in other words, a MidiUnavailableException is thrown by the call to getSequencer().



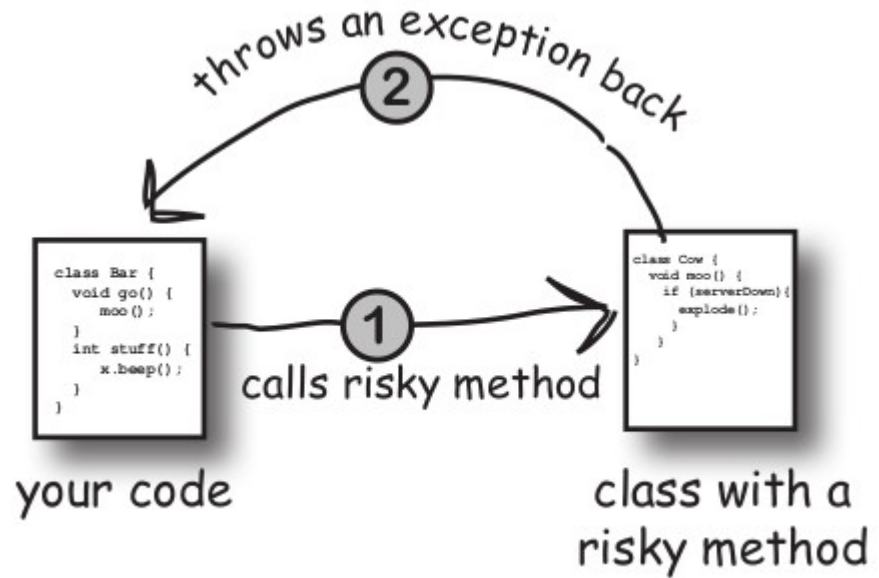
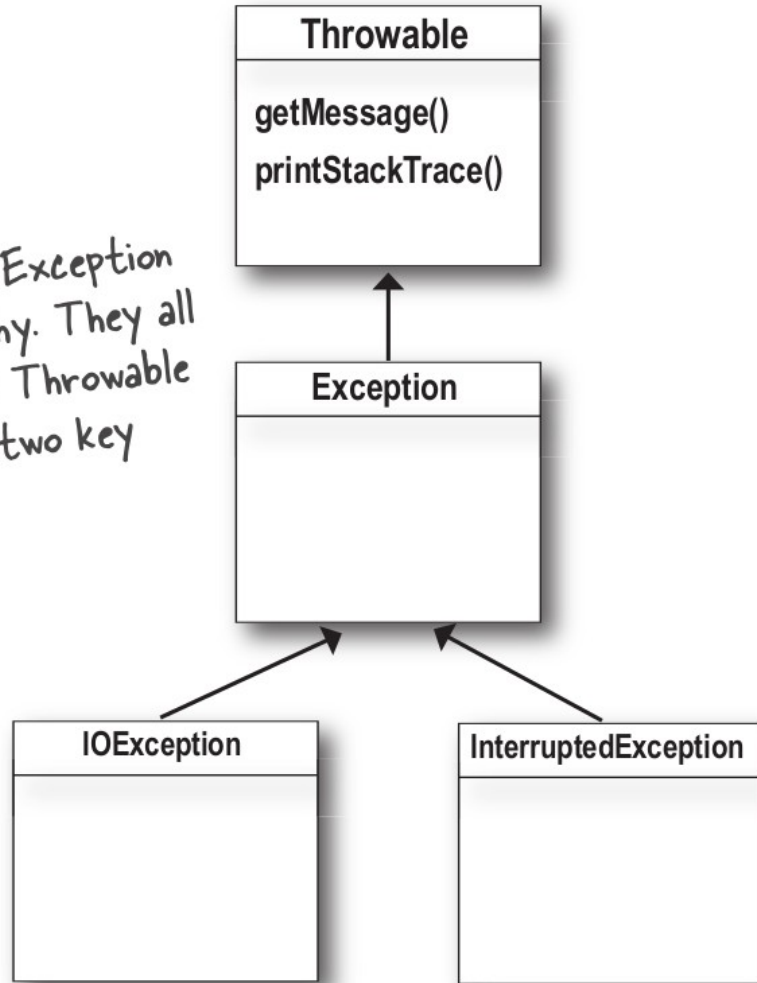
I'm gonna
TRY this risky thing
and I'm gonna
CATCH myself if I fall.

```
try {  
    // do risky thing  
} catch(Exception ex) {  
    // try to recover  
}
```

it's just like declaring
a method argument.

This code only runs if an
Exception is thrown.

Part of the Exception class hierarchy. They all extend class Throwable and inherit two key methods.



① Risky, exception-throwing code:

this method **MUST** tell the world (by declaring) that it throws a `BadException`

```
public void takeRisk() throws BadException {  
    if (abandonAllHope) {  
        throw new BadException();  
    }  
}
```

create a new `Exception` object and throw it.

② Your code that *calls* the risky method:

```
public void crossFingers() {  
    try {  
        anObject.takeRisk();  
    } catch (BadException ex) {  
        System.out.println("Aaargh!");  
        ex.printStackTrace();  
    }  
}
```

← If you can't recover from the exception, at LEAST get a stack trace using the `printStackTrace()` method that all exceptions inherit.

One method will catch what another method throws. An exception is always thrown back to the caller.

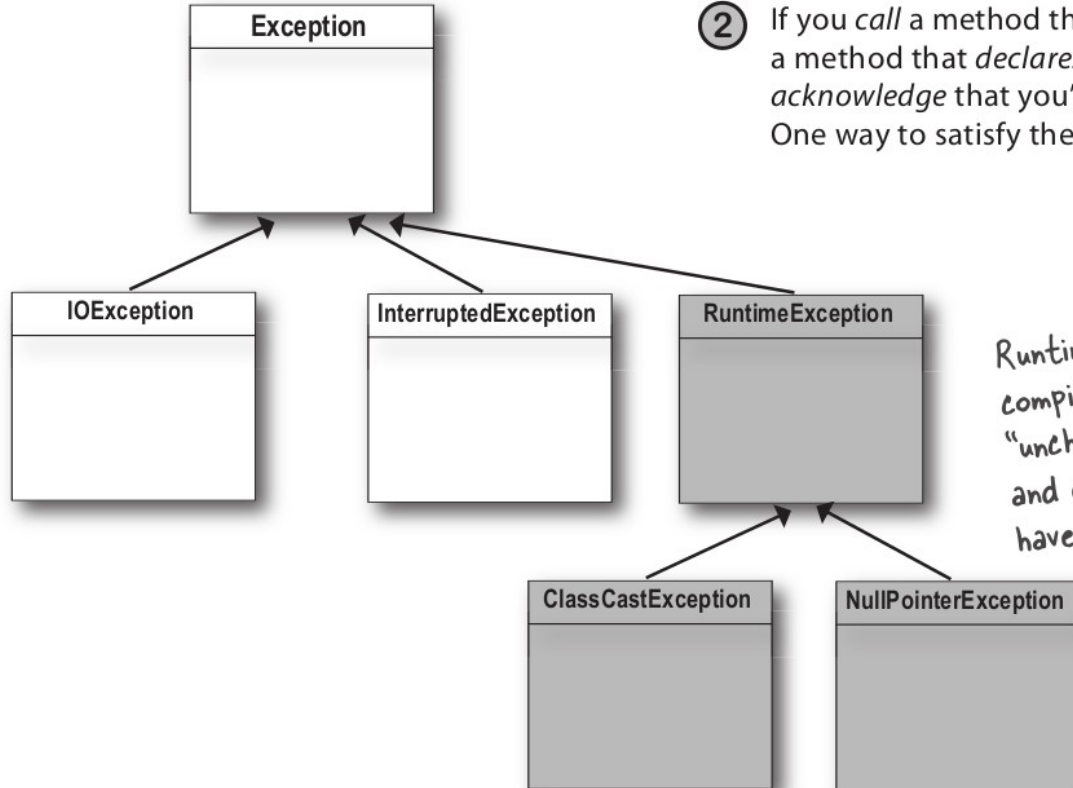
The method that throws has to declare that it might throw the exception.

The compiler checks for everything except RuntimeExceptions.

The compiler guarantees:

- ① If you *throw* an exception in your code you *must* declare it using the *throws* keyword in your method declaration.
- ② If you *call* a method that throws an exception (in other words, a method that *declares* it throws an exception), you must *acknowledge* that you're aware of the exception possibility. One way to satisfy the compiler is to wrap the call in a try/catch.

Exceptions that are *NOT* subclasses of RuntimeException are checked for by the compiler. They're called "checked exceptions"



RuntimeExceptions are *NOT* checked by the compiler. They're known as (big surprise here) "unchecked exceptions". You can throw, catch, and declare RuntimeExceptions, but you don't have to, and the compiler won't check.

If the try succeeds

(doRiskyThing() does *not* throw an exception)

```
try {
```

```
① Foo f = x.doRiskyThing();  
   int b = f.getNum();
```

First the try block runs,
then the code below the
catch runs.

```
} catch (Exception ex) {  
    System.out.println("failed");  
}
```

The code in the
catch block never
runs.

```
② System.out.println("We made it!");
```

File Edit Window Help RiskAll

```
%java Tester
```

```
We made it!
```

If the try fails

(because `doRiskyThing()`
does throw an exception)

The try block runs, but the
call to `doRiskyThing()` throws
an exception, so the rest of
the try block doesn't run.
The catch block runs, then
the method continues on.

```
try {  
  ① Foo f = x.doRiskyThing();  
    int b = f.getNum();  
} catch (Exception ex) {  
  ② System.out.println("failed");  
}  
  ③ System.out.println("We made it!");
```

The rest of the try block never runs, which is a Good Thing because the rest of the try depends on the success of the call to `doRiskyThing()`.

File Edit Window Help RiskAll

%java Tester

failed

We made it!

A finally block is where you put code that must run *regardless* of an exception.

```
try {  
    turnOvenOn();  
    x.bake();  
} catch (BakingException ex) {  
    ex.printStackTrace();  
} finally {  
    turnOvenOff();  
}
```





```
public class Laundry {  
    public void doLaundry() throws PantsException, LingerieException {  
        // code that could throw either exception  
    }  
}
```

This method declares two, count 'em, TWO exceptions.

```
public class Foo {  
    public void go() {  
        Laundry laundry = new Laundry();  
  
        try {  
            laundry.doLaundry();  
        } catch(PantsException pex) {  
            // recovery code  
        } catch(LingerieException lex) {  
            // recovery code  
        }  
    }  
}
```

if doLaundry() throws a **PantsException**, it lands in the **PantsException** catch block.

if doLaundry() throws a **LingerieException**, it lands in the **LingerieException** catch block.

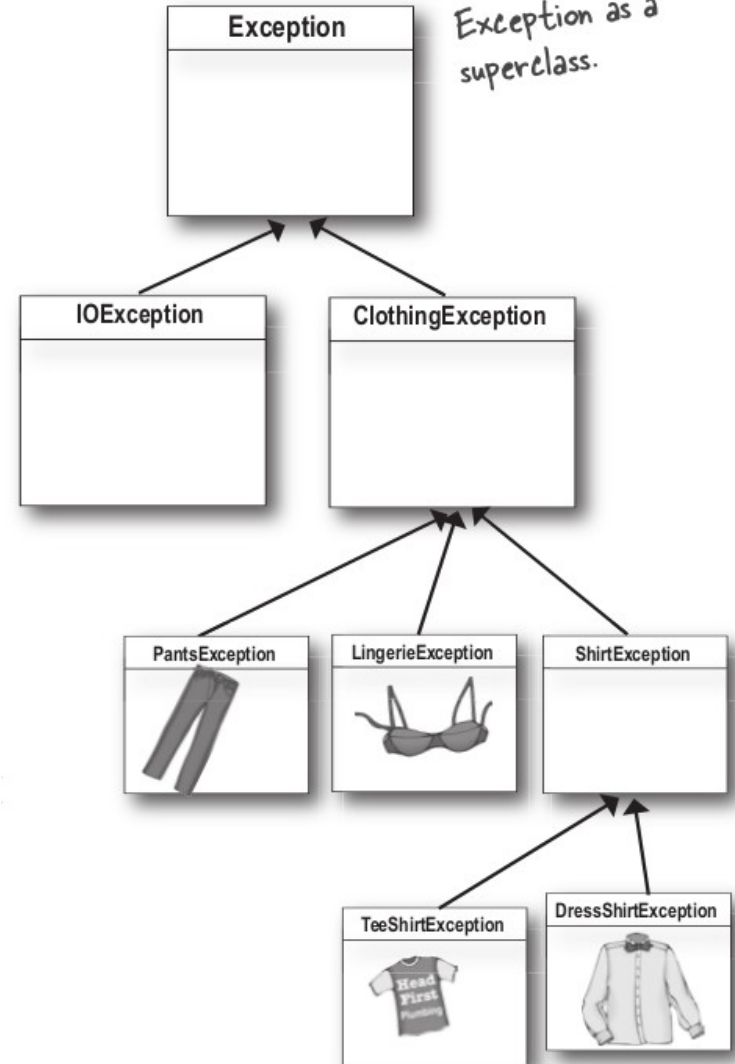
Exceptions are polymorphic

- ① You can **DECLARE** exceptions using a supertype of the exceptions you throw.



```
public void doLaundry() throws ClothingException {
```

Declaring a `ClothingException` lets you throw any subclass of `ClothingException`. That means `doLaundry()` can throw a `PantsException`, `LingerieException`, `TeeShirtException`, and `DressShirtException` without explicitly declaring them individually.



② You can **CATCH** exceptions using a superclass of the exception thrown.

```
try {  
    laundry.doLaundry();
```



Can catch any
ClothingException
subclass

```
} catch(ClothingException cex) {  
    // recovery code  
}
```

```
try {  
    laundry.doLaundry();
```



Can catch only
TeeShirtException and
DressShirtException

```
} catch(ShirtException shex) {  
    // recovery code  
}
```

Just because you CAN catch everything with one big super polymorphic catch, doesn't always mean you SHOULD.

```
try {  
    laundry.doLaundry();  
} catch (Exception ex) {  
    // recovery code...  
}
```

← Recovery from WHAT? This catch block will catch ANY and all exceptions, so you won't automatically know what went wrong.

```
try {  
    laundry.doLaundry();
```



```
} catch (TeeShirtException tex) {  
    // recovery from TeeShirtException
```



```
} catch (LingerieException lex) {  
    // recovery from LingerieException
```



```
} catch (ClothingException cex) {  
    // recovery from all others  
}
```

TeeShirtExceptions and LingerieExceptions need different recovery code, so you should use different catch blocks.

All other ClothingExceptions are caught here.



TeeShirtExceptions are caught here, but no other exceptions will fit.

`catch(TeeShirtException tex)`

Multiple catch blocks must be ordered from smallest to biggest



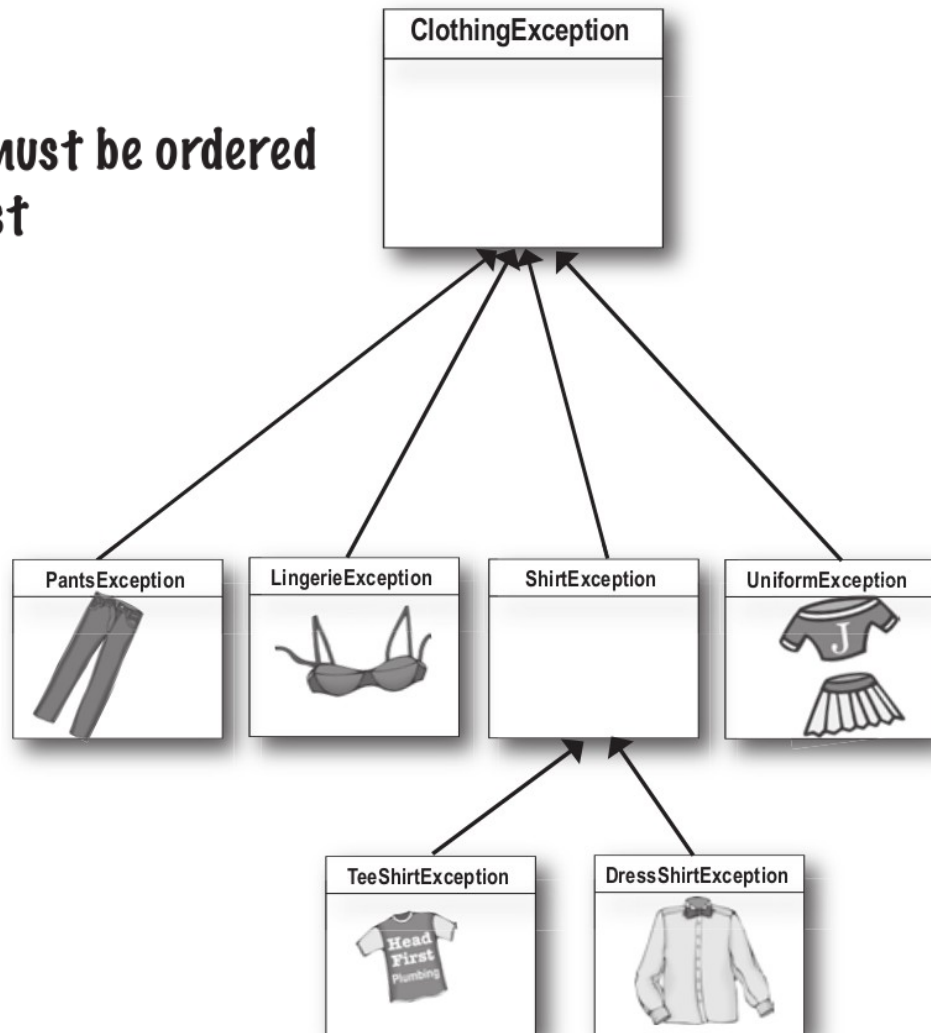
TeeShirtExceptions will never get here, but all other ShirtException subclasses are caught here.

`catch(ShirtException sex)`



All ClothingExceptions are caught here, although TeeShirtException and ShirtException will never get this far.

`catch(ClothingException cex)`



Don't do this!

```
try {  
    laundry.doLaundry();
```



```
} catch(ClothingException cex) {  
    // recovery from ClothingException
```



```
} catch(LingerieException lex) {  
    // recovery from LingerieException
```



```
} catch(ShirtException shex) {  
    // recovery from ShirtException  
}
```

Size matters when you have multiple catch blocks. The one with the biggest basket has to be on the bottom. Otherwise, the ones with smaller baskets are useless.



When you don't want to handle
an exception...

just duck it

```
public void foo() throws ReallyBadException {  
    // call risky method without a try/catch  
    laundry.doLaundry();  
}
```



What the...?

There is NO way I'm
catching that thing. I'm gettin'
out of the way-- somebody
behind me can handle it.

← You don't REALLY throw it, but
since you don't have a try/catch
for the risky method you call,
YOU are now the "risky method".
Because now, whoever calls YOU
has to deal with the exception.

Ducking (by declaring) only delays the inevitable

Sooner or later, somebody has to deal with it. But what if *main()* ducks the exception?

```
public class Washer {  
    Laundry laundry = new Laundry();  
  
    public void foo() throws ClothingException {  
        laundry.doLaundry();  
    }  
  
    public static void main (String[] args) throws ClothingException {  
        Washer a = new Washer();  
        a.foo();  
    }  
}
```

Both methods duck the exception (by declaring it) so there's nobody to handle it! This compiles just fine.



1

doLaundry() throws a
ClothingException



main() calls foo()

foo() calls doLaundry()

doLaundry() is
running and throws a
ClothingException

2

foo() ducks the
exception



doLaundry() pops off the
stack immediately, and
the exception is thrown
back to foo().

But foo() doesn't have a
try/catch, so...

3

main() ducks the
exception



foo() pops off the
stack, and the excep-
tion is thrown back
to main(). But main()
doesn't have a try/
catch, so the excep-
tion is thrown back to...
who? What? There's
nobody left but the
JVM, and it's thinking,
"Don't expect ME to
get you out of this."

4

The JVM
shuts down

Exception Rules

- ① **You cannot have a catch or finally without a try**

```
void go() {  
    Foo f = new Foo();  
    f.foof();  
    catch(FooException ex) { }  
}
```

*NOT LEGAL!
Where's the try?*

- ② **You cannot put code between the try and the catch**

```
try {  
    x.doStuff();  
}  
int y = 43;  
} catch(Exception ex) { }
```

*NOT LEGAL! You can't put
code between the try and
the catch.*

- ③ **A try MUST be followed by either a catch or a finally**

```
try {  
    x.doStuff();  
} finally {  
    // cleanup  
}
```

*LEGAL because you
have a finally, even
though there's no catch.
But you cannot have a
try by itself.*

- ④ **A try with only a finally (no catch) must still declare the exception.**

```
void go() throws FooException {  
    try {  
        x.doStuff();  
    } finally { }  
}
```

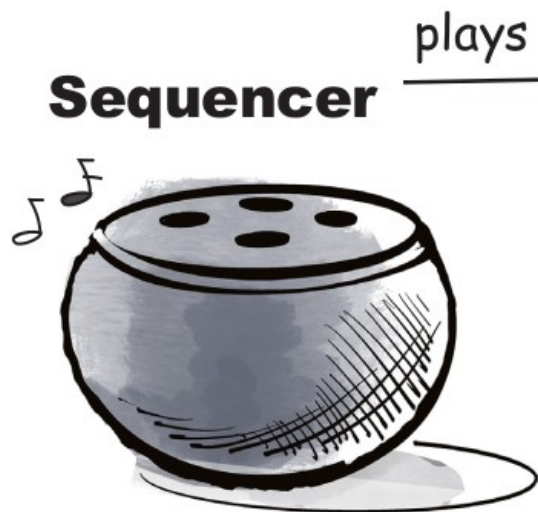
*A try without a catch
doesn't satisfy the
handle or declare law*

① The thing that plays the music

② The music to be played...a song.

③ The part of the Sequence that holds the actual information

④ The actual music information: notes to play, how long, etc.



The Sequencer is the thing that actually causes a song to be played. Think of it like a **smart speaker streaming music**.

plays

Sequence

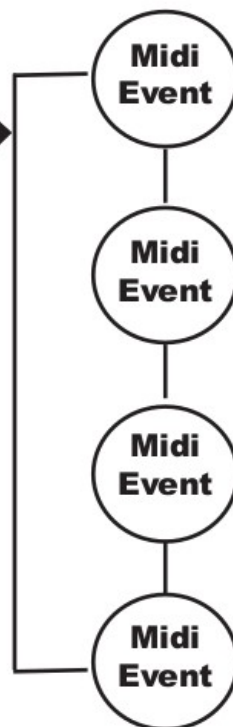


The Sequence is **the song**, the single piece of music that the Sequencer will play.

has a

Track

holds



- ① Get a **Sequencer** and open it

```
Sequencer player = MidiSystem.getSequencer();  
player.open();
```

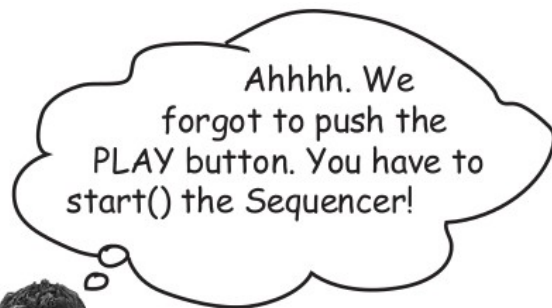
- ② Make a new **Sequence**

```
Sequence seq = new Sequence(timing, 4);
```

- ③ Get a new **Track** from the Sequence

```
Track t = seq.createTrack();
```

- ④ Fill the Track with **MidiEvents** and give the Sequence to the Sequencer



```
player.start();
```


① Make a **Message**

```
ShortMessage msg = new ShortMessage();
```

② Put the **Instruction** in the Message

```
msg.setMessage(144, 1, 44, 100);
```

← This message says, "start playing note 44" (we'll look at the other numbers on the next page).

③ Make a new **MidiEvent** using the Message

```
MidiEvent noteOn = new MidiEvent(a, 1);
```

← The instructions are in the message, but the *MidiEvent* adds the moment in time when the instruction should be triggered. This *MidiEvent* says to trigger message 'a' at the first beat (beat 1).

④ Add the **MidiEvent** to the **Track**

```
track.add(noteOn);
```

← A *Track* holds all the *MidiEvent* objects. The *Sequence* organizes them according to when each event is supposed to happen, and then the *Sequencer* plays them back in that order. You can have lots of events happening at the exact same moment in time. For example, you might want two notes played simultaneously, or even different instruments playing different sounds at the same time.

```
msg.setMessage(144, 1, 44, 100);
```

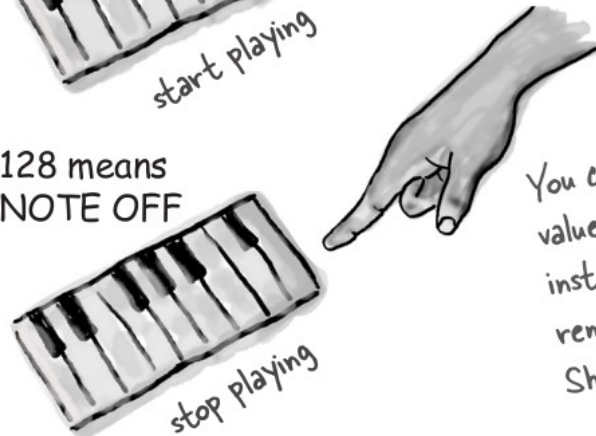
The last 3 args vary depending on the message type. This is a **NOTE ON** message, so the other args are for things the Sequencer needs to know in order to play a note.

① Message type

144 means
NOTE ON



128 means
NOTE OFF



You can use the constant values in `ShortMessage` instead of having to remember the numbers, e.g., `ShortMessage.NOTE_ON`.

② Channel

Think of a channel like a musician in a band. Channel 1 is musician 1 (the keyboard player), channel 9 is the drummer, etc.

③ Note to play

A number from 0 to 127, going from low to high notes.



④ Velocity

How fast and hard did you press the key? 0 is so soft you probably won't hear anything, but 100 is a good default.

Change the instrument

Add a new message, BEFORE the note-playing message, that sets the instrument in channel 1 to something other than the default piano. The change-instrument message is "192," and the third argument represents the actual instrument (try a number between 0 and 127).

```
first.setMessage(192, 1, 102, 0);
```

change-instrument message
in channel 1 (musician 1)
to instrument 102





TRUE OR FALSE

1. A try block must be followed by a catch and a finally block.
2. If you write a method that might cause a compiler-checked exception, you must wrap that risky code in a try/catch block.
3. Catch blocks can be polymorphic.
4. Only “compiler checked” exceptions can be caught.
5. If you define a try/catch block, a matching finally block is optional.
6. If you define a try block, you can pair it with a matching catch or finally block, or both
7. If you write a method that declares that it can throw a compiler-checked exception, you must also wrap the exception throwing code in a try/catch block.
8. The main() method in your program must handle all unhandled exceptions thrown to it.



👍 TRUE OR FALSE 👎

1. A try block must be followed by a catch and a finally block. false
2. If you write a method that might cause a compiler-checked exception, you must wrap that risky code in a try/catch block. false
3. Catch blocks can be polymorphic. true
4. Only “compiler checked” exceptions can be caught. false
5. If you define a try/catch block, a matching finally block is optional. true
6. If you define a try block, you can pair it with a matching catch or finally block, or both true
7. If you write a method that declares that it can throw a compiler-checked exception, you must also wrap the exception throwing code in a try/catch block. false
8. The main() method in your program must handle all unhandled exceptions thrown to it. false

9. A single try block can have many different catch blocks.
10. A method can throw only one kind of exception.
11. A finally block will run regardless of whether an exception is thrown.
12. A finally block can exist without a try block.
13. A try block can exist by itself, without a catch block or a finally block.
14. Handling an exception is sometimes referred to as “ducking.”
15. The order of catch blocks never matters.
16. A method with a try block and a finally block can optionally declare a checked exception.
17. Runtime exceptions must be handled or declared.

- 9. A single try block can have many different catch blocks. true
- 10. A method can throw only one kind of exception. false
- 11. A finally block will run regardless of whether an exception is thrown. true
- 12. A finally block can exist without a try block. false
- 13. A try block can exist by itself, without a catch block or a finally block. false
- 14. Handling an exception is sometimes referred to as “ducking.” false
- 15. The order of catch blocks never matters. false
- 16. A method with a try block and a finally block can optionally declare a checked exception. false
- 17. Runtime exceptions must be handled or declared. false